Object Oriented Programming

Java

File

- A file is a **collection of data** in mass storage.
- A data file is not a part of a program's source code.
- The same file can be read or modified by different programs.
- The program must be aware of the **format** of the data in the file.
- The files are maintained by the operating system.
- The operating system also provides **basic functions** which are called from programs for reading and writing directories and files.

Text File

A computer user distinguishes text (“ASCII”) files and “binary” files. This distinction is based on how you treat the file.

- A text file is assumed to contain lines of text.
- Each line terminates with a new line character.
- Text file cannot store graphical data.
- For example: asd.txt, text.java, html documents, file.dat.

Binary File

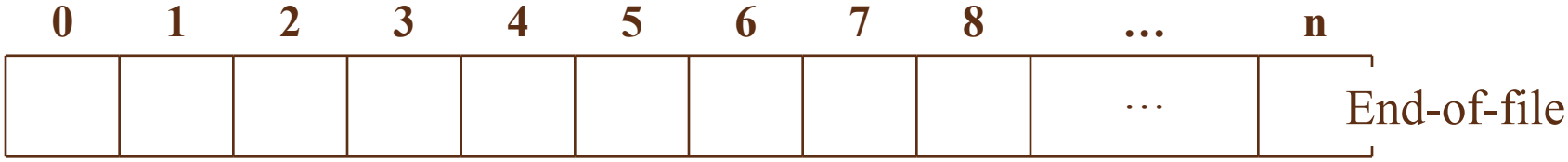
- A “binary” file can contain any information, any combination of bytes.
- It can store text, graphics and sound data in binary format.
- Only a programmer / designer knows how to interpret it.
- Different programs may interpret the same file differently (for example, one program displays an image, another extracts an encrypted message).
- For example: text.class, image.gif, sound.mp3

Stream

- A stream is an **abstract representation** of an input or output device that is a source of or destination for data. You can write data to a stream and read data from a stream.
- A stream is linked to a physical device by the Java I/O system.
- All streams behave in the same manner, even if the actual physical devices they are linked to differ.
- Java implements streams within class hierarchies defined in the **java.io** package.

File and Stream

- Java views a file as a stream of bytes.
- File ends with end-of-file marker or a specific byte number.
- Java file I/O involves streams. You write and read data to streams.



- Three **stream objects** are automatically created for every application:
  - System.in
  - System.out
  - System.err

Byte Stream and Character Stream

- There are two types of streams:
  - Byte stream
  - Character stream
- Byte streams create binary files.
- They store bits as they are in memory.
- Binary files are faster to read and write because no translation need take place.
- Binary files, however, cannot be read with a text editor.

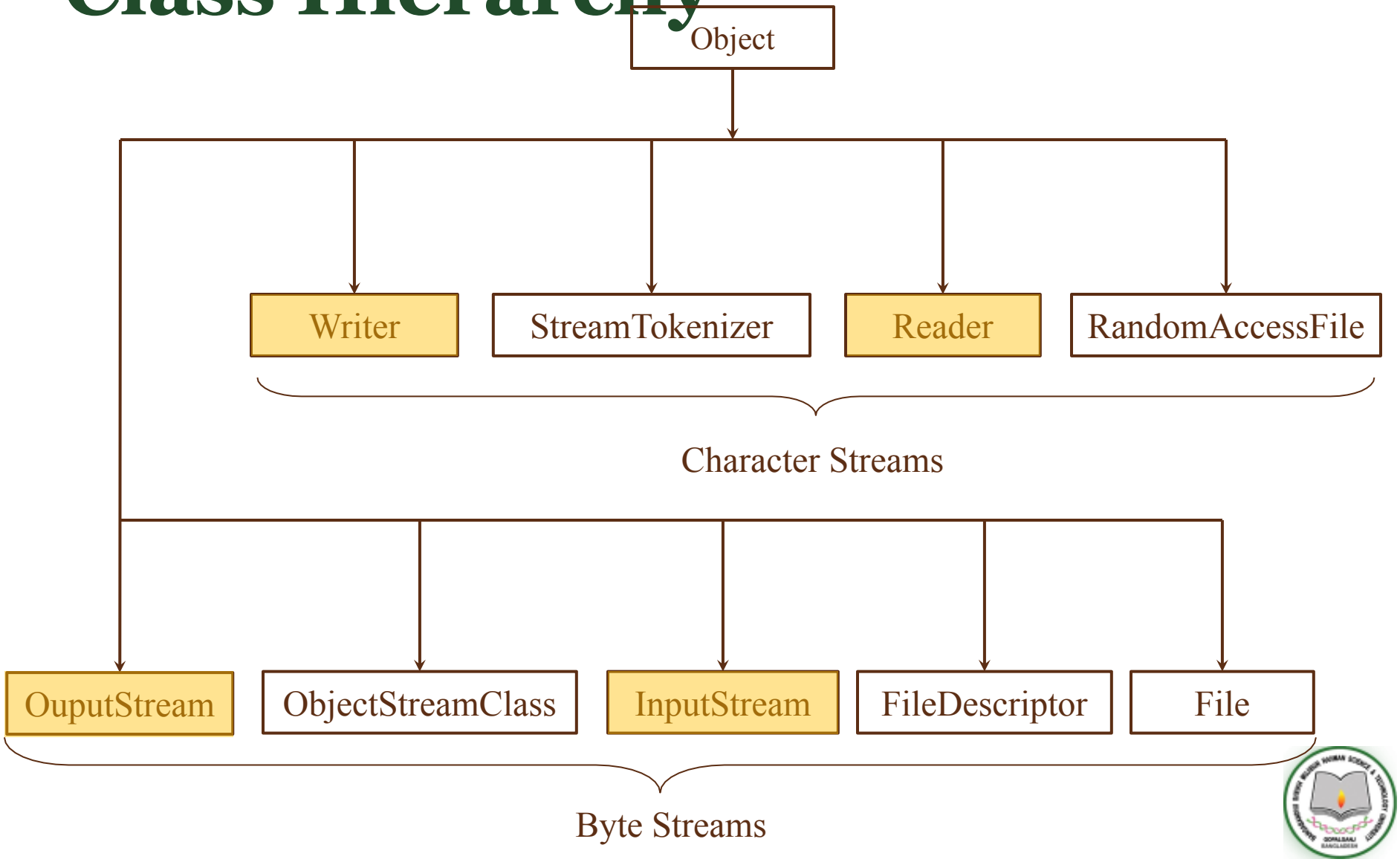
Byte Stream and Character Stream

- Character streams create text files.
- These are files designed to be read with a text editor.
- Java automatically converts its internal unicode characters to the local machine representation (ASCII).

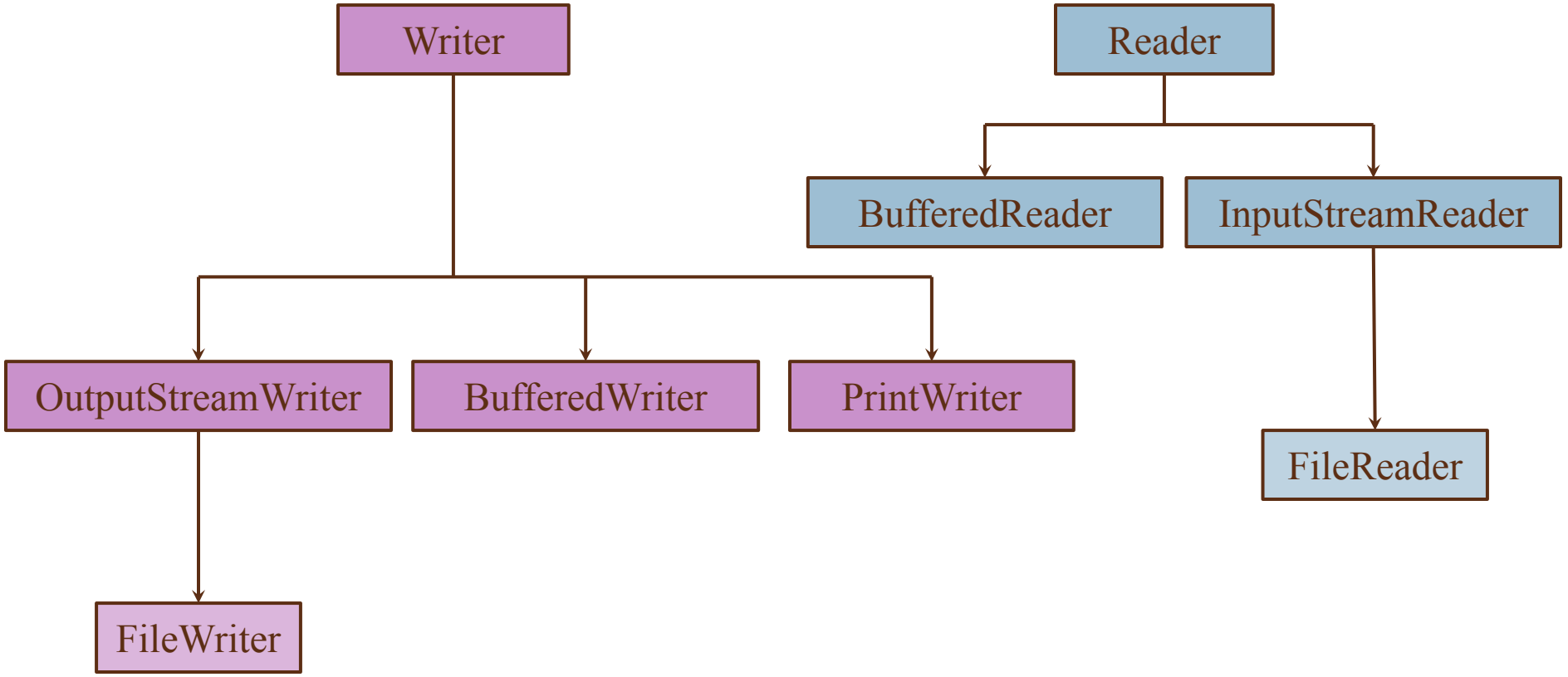
Stream in Java

- Input stream: a stream that provides input to a program
  - System.in is an input stream.
- Output stream: a stream that accepts output from a program
  - System.out is an output stream
- A stream connects a program to an I/O object
  - System.out connects a program to the screen
  - System.in connects a program to the keyboard.

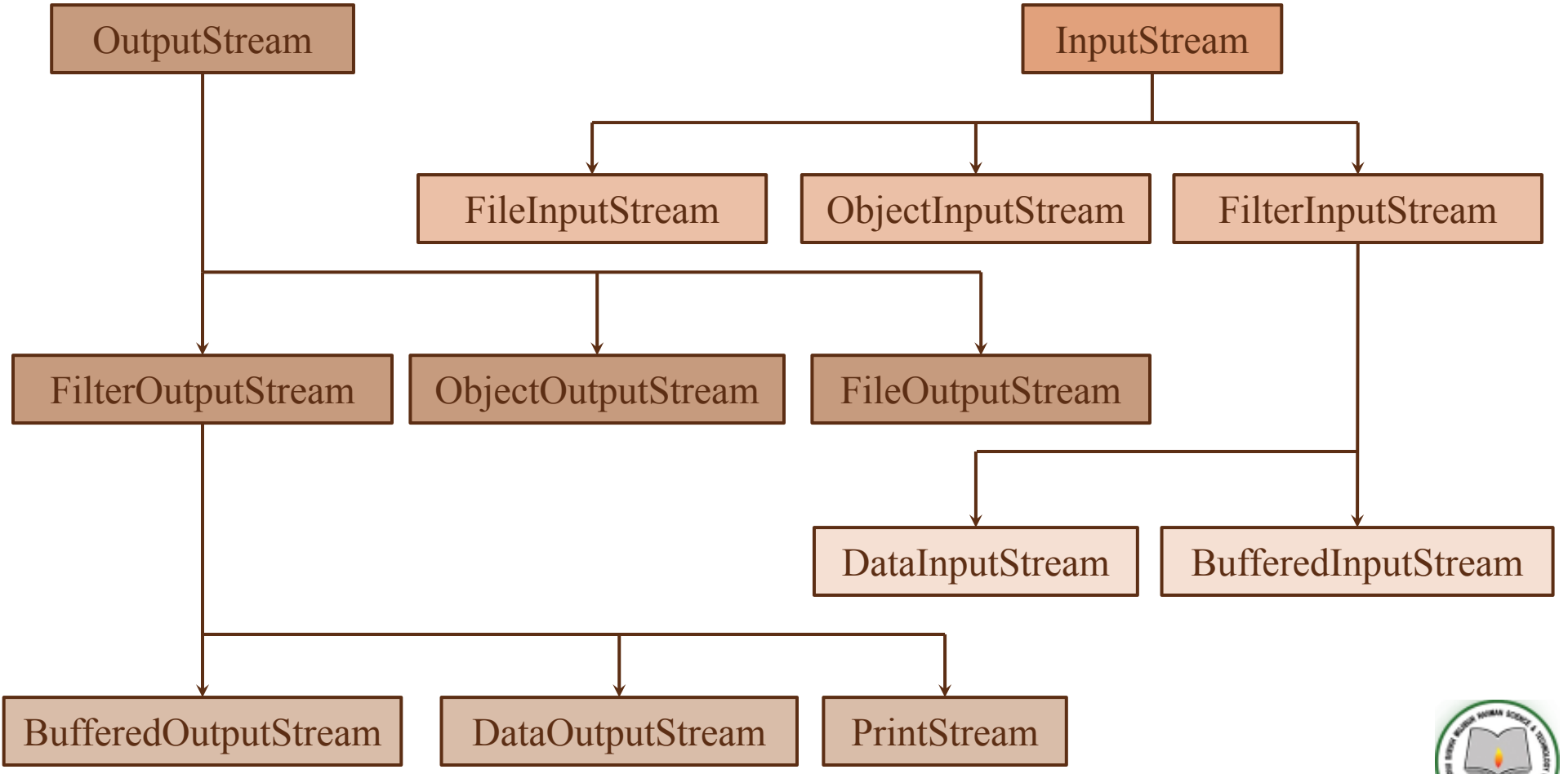
Class Hierarchy



Class Hierarchy



Class Hierarchy



Reading Console Input

- Java input console is accomplished by reading from **System.in**.
- To obtain a character-based stream that is attached to the console, you wrap System.in in a BufferedReader object, to create a character stream.
- Most common syntax to obtain BufferedReader:

```
BufferedReader br = new BufferedReader(new
InputStreamReader(System.in));
```

- Once BufferedReader is obtained, we can use read( ) method to read a character or readLine( ) method to read a string from the console.

Writing Console Onput

- Console output is most easily accomplished by print( ) and println ( ).
- These methods are defined by the class **PrintStream** which is the type of the object referenced by **System.out**.
- Because PrintStream is an output stream derived from OutputStream, it also implements the low-level method write( ). Thus, write( ) can be used to write to the console.

```
byte b = 'A';
System.out.write(b);
```

BufferedReader vs Scanner

BufferedReader	Scanner
BufferedReader only reads data	Scanner reads and parses data
The Buffer size of BufferedReader is large(8KB)	The Buffer size of scanner is small(1KB)
BufferedReader is more suitable for reading file with long String	Scanner is more suitable for reading small user input from command prompt.
BufferedReader is synchronized among multiple threads	Scanner is not synchronized which means you cannot share Scanner among multiple threads.
BufferedReader is faster	Scanner is slower as it spends time on parsing.
BufferedReader is from java.io package	Scanner is from java.util package

Reading and Writing Files

- The following takes a file object to create an input stream object to read the file.
- First we create a file object using File() method as follows:

```
File f = new File("C:/Desktop/first.txt");
InputStream fis = new FileInputStream(f);
```

- Following takes a file object to create an output stream object to write the file.
- First we create a file object using File() method as follows:

```
File f = new File("C:/Desktop/first.txt");
OutputStream fos = new FileOutputStream(f);
```

Reading and Writing Files

```
public class FileTest {
    public static void main(String[] args){
        File f = new File("C:/Users/pc/Desktop/first.txt");
        InputStream fis = new FileInputStream(f);

        File ff = new File("C:/Users/pc/Desktop/second.txt");
        OutputStream fos = new FileOutputStream(ff);

        while(fis.available() > 0){
            char c = (char) fis.read();
            fos.write(c);
        }
    }
}
```

This program reads characters from one file and writes them in another file.

Methods of FileInputStream

Method	Description
int available()	It is used to return the estimated number of bytes that can be read from the input stream.
int read()	It is used to read the byte of data from the input stream.
int read(byte[] b)	It is used to read up to **b.length** bytes of data from the input stream.
protected void finalize()	It is used to ensure that the close method is called when there is no more reference to the file input stream.
void close()	It is used to closes the stream